



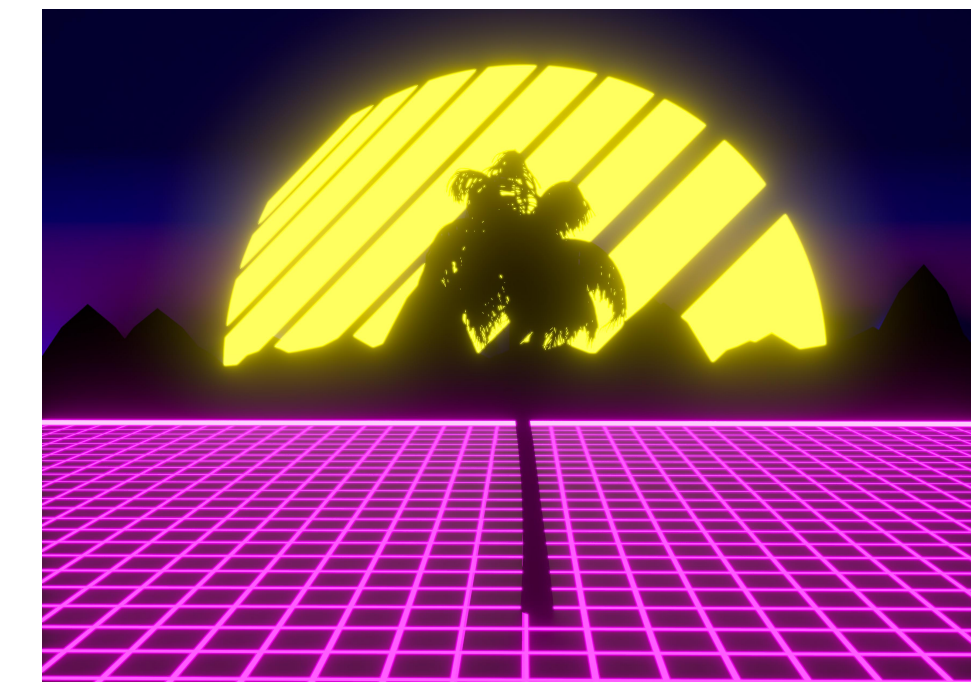
UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

VAPOR-WAVE GRAPHICS, DOMAIN SPECIFIC LANGUAGE

Helberth Garcia. Autor^{1*}

²Universidad Distrital Fransisco Jose de Caldas
*hagarciam@udistrital.edu.co

ID: EVNT2025-0001



INTRODUCTION

his poster presents the development of a domain-specific language (DSL) for generating images in the vaporwave aesthetic. Despite the popularity of visual DSLs in technical fields, few are tailored for creative or nostalgic styles. This project bridges compiler construction with digital art, proposing a simple grammar for artistic expression through code.

Key motivations:

- Explore the intersection of programming languages and visual art;
- Provide a minimalist and nostalgic tool for image generation;
- Develop a prototype focused on clarity and extensibility.

GOAL

To design and implement a DSL that allows users to generate vaporwave-style graphics through declarative scripts. The expected result is a working prototype that translates code into aesthetic visual compositions.

LITERATURE REVIEW

Previous works have explored domain-specific languages (DSLs) in both technical and creative contexts. For instance, Hudak (1996) emphasizes how DSLs can effectively capture and formalize domain knowledge by embedding the logic and semantics of specific problem areas directly into the language. This approach enhances expressiveness and abstraction, making DSLs powerful tools in software engineering and system design. On the other hand, McCormack (2005) explores how generative systems, including DSLs, can be harnessed for artistic purposes—such as in digital art and creative coding—by enabling non-traditional outputs and aesthetics that challenge the boundaries between code and artistic expression.

EXPERIMENTS

Five scripts with increasing complexity were executed to evaluate syntax coverage, rendering output, and error handling. Scripts introduced gradients, shapes, retro symbols, and visual filters. Robustness was preliminarily assessed via malformed inputs.

PROPOSED SOLUTION

The DSL is implemented in Python using a define generative grammar. A lexer, parser, and simple renderer (via Pillow) translate text commands into graphical outputs. The architecture promotes extensibility.

RESULTS AND DISCUSSION

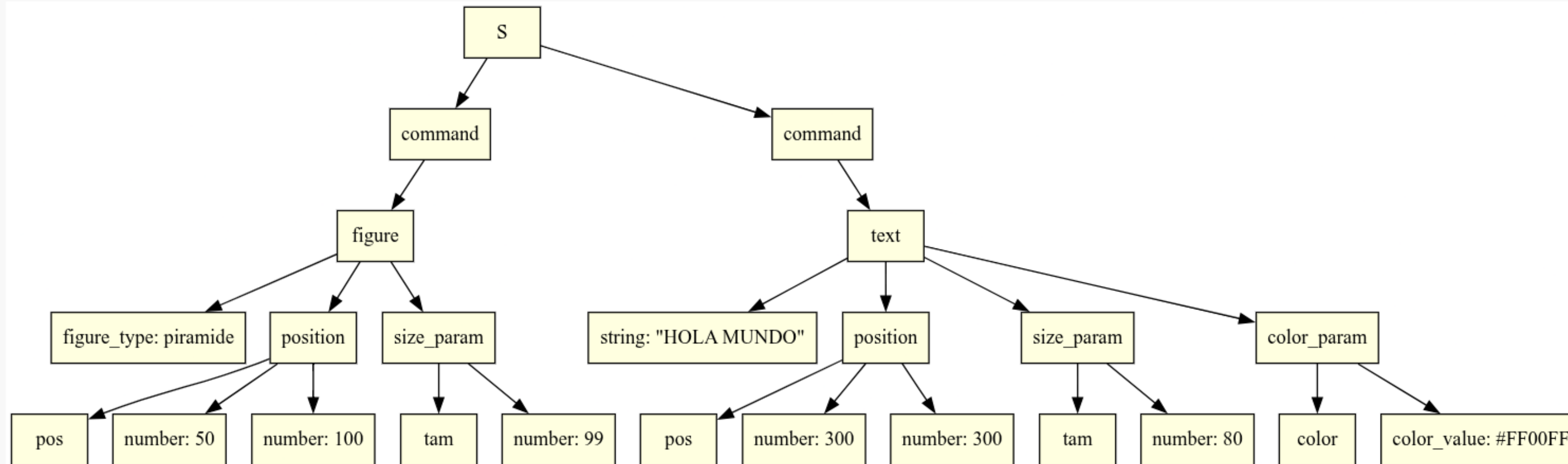
Outputs showed correct visual rendering for all valid scripts. Syntax errors triggered helpful feedback. Visuals adhered to the vaporwave style, and the language was intuitive to use.

Figura 1 – Sample image generated with the DSL



Fonte: own work (2025)

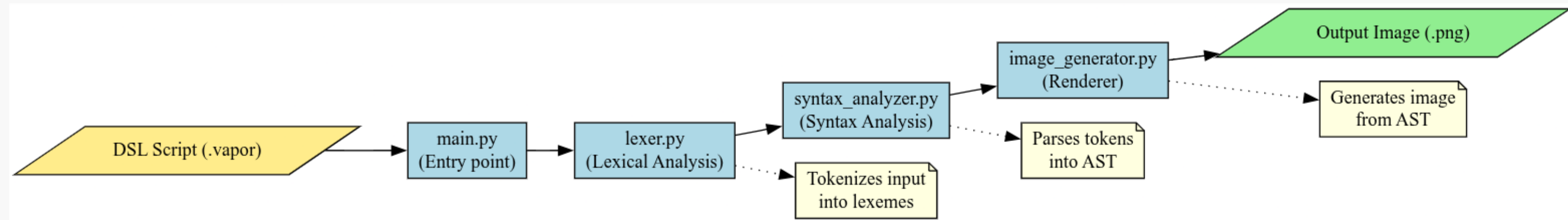
Figura 2 – Syntax Tree Example



Fonte: own work (2025)

RESULTS AND DISCUSSION (CONTINUED)

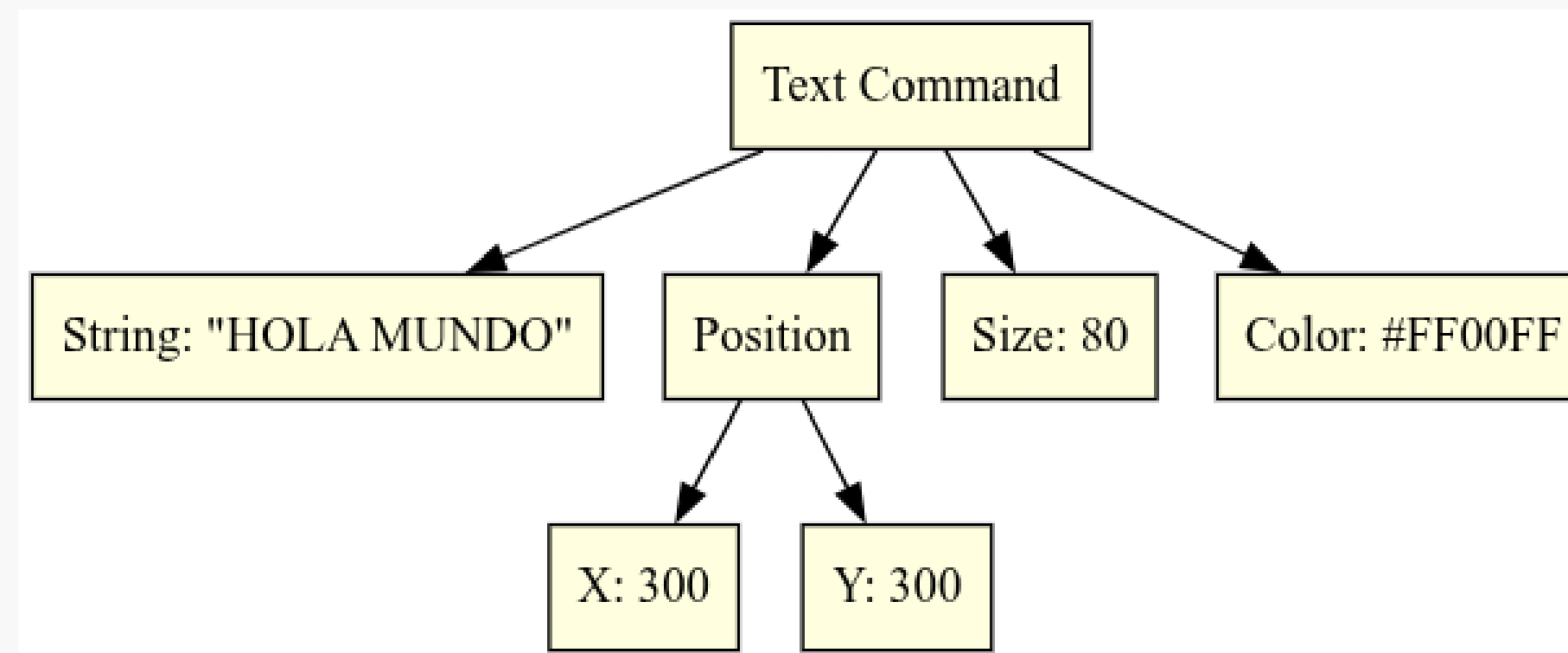
Figura 3 – DSL Compilation Pipeline



Fonte: own work (2025)

This diagram shows the core architecture: user scripts are parsed into tokens, analyzed syntactically, and compiled into drawing commands using a custom Python renderer based on Pillow. Each stage is modular and open for future enhancements like optimization or animation support.

Figura 4 – Abstract Syntax Tree (AST)



Fonte: own work (2025)

The compiler builds an AST from user input. This tree structure enables flexible and accurate translation from code to visuals, and supports recursive composition of commands (e.g., grouped elements or sequences).

CONCLUSIONS

- Designed and tested a DSL tailored to vaporwave-style graphics.
- Demonstrated the feasibility of compiler principles in artistic domains.
- Laid groundwork for future extensions, such as animations and user-defined styles.

REFERENCES

- HUDAK, Paul. Building domain-specific embedded languages. **ACM Computing Surveys**, 1996.
- MCCORMACK, Jon. Generative art and creative systems. **Proceedings of the Second Iteration Conference**, 2005.